

자기소개서

JAEWOO CHOI · GAME PROGRAMMER

010-2776-2295 wod6983@gmail.com <https://jaw-choi.github.io/> <https://github.com/jaw-choi>

1. 지원동기

저는 계명대학교와 DigiPen Institute of Technology 복수학위 과정을 거치며, 이론으로 배운 개념은 직접 손으로 구현해 봐야 비로소 제 것이 된다는 확신을 갖게 되었습니다. 그래서 졸업 이후에도 개인 프로젝트를 진행해왔습니다. 콘솔 뱀서라이크, 눈덩이를 굴러라, 그리고 현재 진행 중인 LTF까지, 세 프로젝트는 규모도 형태도 다르지만 공통점이 하나 있습니다. 모두 실제로 동작하는 결과물을 만들었고, 그 과정에서 부딪힌 문제를 회피하지 않고 기록하며 해결해왔다는 점입니다. 게임 프로그래머로서 제가 하고 싶은 일은 이 태도를 그대로 이어가는 것입니다.

2. 강점 — 문제를 대하는 태도

제 강점은 문제를 "일단 되게 만드는 것"과 "왜 이렇게 만들었는지 설명할 수 있는 것" 사이에서 후자를 놓치지 않으려 한다는 점입니다. 콘솔 뱀서라이크를 개발할 때 저는 의사결정과 트러블슈팅 과정을 처음부터 끝까지 문서로 남기는 습관을 들였습니다. 이 습관은 이후 5인이 함께 움직이는 LTF 프로젝트에서, 제가 설계한 전투 시스템을 다른 팀원이 이해하고 그 위에 스킬을 엮을 수 있게 만드는 데 그대로 이어졌습니다. 코드를 "동작하게" 만드는 것과 "다른 사람이 이어받을 수 있게" 만드는 것은 다른 일이라는 걸 반복된 협업 경험을 통해 배웠습니다.

다만 혼자 결정하고 문서화하는 방식에 익숙하다 보니, LTF처럼 여러 명이 함께 움직이는 프로젝트 초반에는 제 방식대로 정리한 내용을 팀원들과 맞춰가는 데 시간이 걸리는 편이었습니다. 지금은 문서를 먼저 팀과 공유하고 피드백을 받아 수정하는 순서로 진행하며 이 부분을 보완하고 있습니다.

3. 직무 경험과 문제 해결

LTF — TFT 스타일 오토배틀러 (Client Programmer / 5인팀, 2026.06 ~ 진행중)

현재 진행 중인 LTF에서 저는 Unreal Engine의 Gameplay Ability System(GAS)을 기반으로 전투 상태머신과 공격 판정 파이프라인을 설계하고, 챔피언 스킬을 Gameplay Ability로 20종 이상 구현했습니다.

가장 기억에 남는 문제는 멀티플레이어 환경에서 평타 데미지가 여러 번 중복 적용되던 버그였습니다. 관전하는 클라이언트 수가 늘어날수록 데미지가 정비례해서 커지는 패턴을 발견했고, 이를 단서로 원인을 추적했습니다. 공격 몽타주는 서버에서 재생을 지시하면 모든 클라이언트에 복제되어 각자 재생되는데, 데미지 판정을 트리거하는 AnimNotify가 복제된 모든 인스턴스에서 동일하게 실행되고 있었습니다. 즉 서버 1개와 클라이언트 N개가 각각 판정 이벤트를 보내고 있었던 것입니다. AnimNotify에 HasAuthority() 체크를 추가해 서버 인스턴스에서만 데미지 이벤트를 전송하도록 수정했고, 이후로는 몽타주·AnimNotify 기반 판정 로직 전반에 관한 검증은 습관적으로 넣게 되었습니다. 이 경험으로 "연출을 위한 복제"와 "결과에 반영되어야 하는 로직"을 구분하는 감각을 체득했습니다.

또한 챔피언 스탯 데이터를 엑셀에서 Unreal DataTable로 가져오는 파이프라인을 관리하면서, 기획이 스탯 값을 조정하거나 새로운 스탯 컬럼(초기 마나 등)을 추가할 때마다 Row 구조체를 확장해 대응했습니다. 코드와 기획 데이터 사이의 경계를 유연하게 유지하는 것이 실제 협업에서 얼마나 중요한지 배울 수 있었습니다.

콘솔 뱀서라이크 (Solo Project, 2026.02)

1인 개발로 일주일간 진행한 프로젝트에서, 화면에 존재하는 모든 오브젝트를 매 프레임 전수 비교하던 충돌 처리 방식이 적 개체 수가 늘어나며 프레임 저하로 이어지는 문제를 마주했습니다. QuadTree로 공간을 분할해 충돌 후보를 범위 내 객체로 한정함으로써 연산 복잡도를 크게 낮췄고, 오브젝트 풀링을 함께 도입해 빈번한 생성·소멸로 인한 메모리 단편화도 방지했습니다. 짧은 개발 기간이었던 만큼 매일의 의사결정과 트러블슈팅을 기록으로 남겼고, 이 습관 덕분에 시간이 지난 뒤에 프로젝트를 다시 열어봐도 왜 그렇게 구현했는지 스스로 설명할 수 있었습니다.

눈덩이를 굴러라! (Game Programmer / 2인팀, 2025.12 ~ 현재, Google Play·App Store 출시)

2인 팀으로 기획부터 출시까지 전 과정을 경험한 프로젝트입니다. 디자이너가 만든 Figma 시안을 유니티 UI로 옮기는 과정에서 반복 작업과 커뮤니케이션 비용이 컸습니다. Figma→Unity 변환 플러그인을 도입하고 네이밍 규칙을 표준화해 이 비용을 줄였고, 배포 과정에서 반복되던 실수를 막기 위해 릴리즈 체크리스트와 버전·코드 규칙을 만들어 운영했습니다. 그 결과 Google Play와 App Store에 실제로 게임을 출시했고, 지금도 후속 업데이트를 이어가고 있습니다. 이 프로젝트를 통해 "기능을 완성하는 것"과 "출시할 수 있는 상태로 만드는 것"은 전혀 다른 일이라는 걸 체감했습니다.

A ROUND ENTERTAINMENT — Software Engineer Intern (2024.01 ~ 2024.04)

데이팅 앱 Joopy의 프로필 편집 화면 개발에 UI/UX 팀과 협업하여 참여했습니다. 이미지 순서 변경이 실시간으로 반영되지 않는 비동기 처리 문제를, async/await, Promise, Firebase 연동 구조를 정리해 해결한 경험이 있습니다. 게임 클라이언트 밖에서도 비동기 처리와 상태 동기화는 결국 같은 종류의 문제라는 것을 확인할 수 있었습니다.

4. 입사 후 포부

입사 후에도 지금까지 해왔던 방식 그대로, 기능을 동작시키는 데서 멈추지 않고 왜 그렇게 동작하는지 파악하고 기록하며 일하고 싶습니다. 특히 멀티플레이어 동기화나 기획 데이터 파이프라인처럼 여러 사람이 함께 건드리는 영역에서, 문제가 재발하지 않도록 원인을 구조로 남기는 역할을 맡고 싶습니다. 혼자 완성도를 높이는 데서 나아가, 제가 만든 시스템 위에서 다른 팀원이 스킬을 엮고 기획자가 수치를 바꿀 수 있도록 만드는 프로그래머로 성장하고 싶습니다.